

LIBERTY PRESS FRAMEWORK — PROMPT MAESTRO

Documento de contexto completo para cualquier IA o desarrollador que tome este proyecto

¿QUÉ ES ESTE PROYECTO?

Liberty Press Framework es un sistema de automatización de prensa política construido sobre n8n. Su propósito comercial es venderse como un "Centro de Operaciones de Prensa 24/7" a intendentes, directores de prensa de municipios y medios de comunicación regionales de Argentina (foco inicial: provincia de Mendoza).

El sistema toma noticias de fuentes locales y nacionales, las procesa con IA (Gemini), las filtra y encuadra según el perfil político del cliente, y las distribuye a portal web + redes sociales con aprobación humana vía Telegram antes de publicar.

No es un script. Es un servicio B2B de comunicación política automatizada.

LÓGICA DE NEGOCIO CENTRAL

El motor político

Cada cliente tiene un "perfil político" que determina:

- Qué noticias amplificar (logros de gestión, críticas al adversario)
- Qué noticias vetar (escándalos propios, conflictos gremiales)
- Cómo encuadrar las noticias nacionales (a favor o en contra según alineación)
- Qué personas proteger y cuáles son "objetivo"

Ejemplos concretos:

- Cliente = intendente peronista de Malargüe → proteger a Jaque, criticar a Cornejo (gobernador radical) y a Milei (nacional LLA)
- Cliente = oposición en Malargüe → amplificar conflictos ATE vs municipio, ignorar el feed oficial de la municipalidad

Regla de oro del contenido

Las noticias importantes que ocurren en el distrito SE PUBLICAN SIEMPRE, independientemente del perfil político (persona desaparecida, catástrofe, alerta climática). Lo que cambia es el ENCUADRE, no la omisión total.

Flujo de aprobación humana (Human-in-the-loop)

Ninguna noticia se publica sin que el director de prensa del cliente apriete un botón en Telegram. El sistema propone, el humano aprueba. Esto es el diferencial de venta: el cliente mantiene el control total.

ARQUITECTURA TÉCNICA

Stack tecnológico

- **Motor de automatización:** n8n (self-hosted, local con Cloudflare Tunnel)
- **IA de procesamiento:** Google Gemini 2.5 Flash
- **Base de datos:** Google Sheets (MVP, escalar a Supabase/Postgres después)
- **Aprobación:** Telegram Bot (inline keyboard con callback_query)
- **Publicación web:** Astro (sitio estático) + GitHub Pages + Cloudflare Pages
- **Publicación redes:** Meta Graph API (Facebook/Instagram), X API
- **Infraestructura:** Cloudflare Tunnel (gratis, URL pública permanente)
- **Imágenes:** Bannerbear o Puppeteer (pendiente de implementar)
- **Scraping redes:** Apify (free tier para demo)

Infraestructura actual del desarrollador

- n8n corriendo en localhost:5678 (Windows, usuario "Ryzen 5")
- Cloudflare Tunnel activo: haven-que-screensaver-collapse.trycloudflare.com (NOTA: Quick Tunnel cambia URL al reiniciar. Para producción usar tunnel permanente con dominio propio en Cloudflare)
- Script de inicio: C:\Users\Ryzen 5\Desktop\iniciar_liberty.ps1
- Archivos de config n8n: C:\Users\Ryzen 5.n8n\
- Proyecto Railway preparado: C:\Users\Ryzen 5\Desktop\n8n-railway (listo para deploy cuando haya primer cliente pagador)
- Sitio de demo activo: lalibertadavanzandohoy.pages.dev (Cloudflare Pages)
- Repositorio GitHub: <https://github.com/andrescarrizo1/lalibertad-web>

Los tres workflows de n8n

Workflow A — Ingesta (cada 20 minutos)

```
Schedule → RSS feeds locales → Agrupar top noticias →  
Gemini (con perfil político) → Limpiar JSON →  
Google Sheets (status: PENDING) → Telegram (botones aprobación)
```

Workflow B — Callback Telegram (event-driven)

Workflow C — Publisher (cada 25 minutos)

Máquina de estados en Google Sheets

Columnas del Google Sheet (base de datos)

ESTRUCTURA DE ARCHIVOS DEL PROYECTO

```
| | | └─ fuentes/
| | |   └─ rss_feeds.txt
| | |   └─ facebook_pages.txt
| | |   └─ instagram_pages.txt
| | |
| | └─ san_carlos/      ← Estructura vacía, próximo target
| | └─ tunuyan/        ← Estructura vacía, próximo target
| | └─ _demo/
| |
| └─ docs/
|   └─ propuesta_comercial/
|       └─ planes_precios.md
|       └─ pitch_deck_notas.md
|   └─ onboarding/
|       └─ checklist_nuevo_cliente.md
|
└─ README.md
```

FUENTES DE CONTENIDO — MALARGÜE (verificadas)

RSS activos (XML confirmado)

Feed	URL	Sesgo	Prioridad
Malargüe a Diario	https://www.malargueadiario.com/feed/	Crítico/político	Alta
Malalweb	https://malalweb.com.ar/feed/	Neutro/variado	Alta
Municipalidad	https://www.malargue.gov.ar/feed/	Oficialismo puro	Media

Redes sociales (requieren Apify para scraping)

Medio	Facebook	Instagram
Malalweb	facebook.com/malalwebdiariodigital	—
Ser y Hacer	facebook.com/SyHMalargue	instagram.com/seryhacerdemalargue
LV19 Radio	facebook.com/radiomalargue	instagram.com/lv19radiomalargue
FM Eólica	facebook.com/eolica.puraenergia	instagram.com/fmeolica

CONTEXTO POLÍTICO — MALARGÜE (mayo 2025)

- **Intendente:** Celso Jaque — Partido Justicialista (Peronismo)
- **Gobernador Mendoza:** Alfredo Cornejo — UCR/Juntos por el Cambio
- **Presidente Argentina:** Javier Milei — La Libertad Avanza
- **Relación Jaque-Cornejo:** CONFLICTO (ver noticia Portezuelo del Viento)
- **Relación Jaque-Milei:** OPOSICIÓN
- **Dinámica local:** Jaque es oposición a provincia y nación. Cualquier cliente pro-Jaque atacará gobierno provincial y nacional. Cualquier cliente opositor a Jaque se alinearé con Cornejo o LLA.

PLANES COMERCIALES

Plan	Posts/día	Precio ARS/mes	Incluye
Presencia	2	\$400.000	Instagram + Facebook + Telegram
Gestión	5	\$700.000	+ Portal web + X + Reporte semanal
Comando	10	\$1.200.000	+ Crisis radar + WhatsApp + Campaña

Add-ons:

- Dominio propio: +\$15.000/mes
- Modo campaña electoral: +\$300.000/mes
- Scraping redes sociales (Apify): +\$80.000/mes
- WhatsApp Business broadcast: +\$150.000/mes

Filosofía de precio: El sistema reemplaza a un community manager político medio tiempo (\$150k-400k/mes) con mejor resultado, disponibilidad 24/7 y cobertura de todas las redes simultáneamente.

ROADMAP — ESTADO ACTUAL Y PRÓXIMOS PASOS

✅ Completado

- Workflow de ingesta (RSS → Gemini → Sheets → Telegram)
- Máquina de estados (PENDING→APROBADO→PROCESANDO→PUBLICADO→ERROR)
- Webhook Telegram para aprobación (callback_query)
- Parser robusto de Gemini (con fallbacks y strip de markdown fences)
- Push a GitHub con manejo de errores (continueOnFail + IF Exitó?)
- Credenciales Google Sheets unificadas (todas usan account 6)
- Cloudflare Tunnel activo (n8n accesible desde internet)
- Script de inicio unificado (iniciar_liberty.ps1)
- Estructura de carpetas del framework
- Config político de Malargüe (pro_gobierno + oposicion)
- Fuentes RSS de Malargüe verificadas (3 feeds activos)

🔄 En progreso

- Demo de Malargüe funcionando end-to-end
- Verificación del flujo completo (Telegram → Sheets → GitHub → Cloudflare Pages)

📋 Pendiente (en orden de prioridad)

1. **Generación automática de imágenes** (Bannerbear o Puppeteer) Elimina el último paso manual (abrir VS Code para la imagen destacada)
 2. **Publicación en Instagram/Facebook** (Meta Graph API) Después del push a GitHub, publicar simultáneamente en redes Flujo Instagram: crear container → publicar (2 pasos obligatorios) Requisito: imagen en URL pública ANTES de llamar a la API
 3. **Arquitectura multi-tenant** (un config, múltiples clientes) Sheet maestro con credenciales por client_id El workflow lee el config dinámicamente al iniciar
 4. **Radar de crisis** (social listening) Monitoreo de menciones negativas en tiempo real vía Apify Alerta Telegram en < 5 minutos con borrador de respuesta
 5. **Script de generación de nuevo cliente** Antigravity lee config_cliente.json y genera workflow + carpeta en menos de 2 minutos
 6. **Deploy en Railway** (cuando haya primer cliente) Archivos ya preparados en Desktop/n8n-railway/ Migrar de localhost a servidor en la nube
-

CÓMO AGREGAR UN NUEVO CLIENTE (checklist operativo)

1. Copiar `_template/` a `clientes/[nombre_municipio]/`
 2. Completar `config.json` (partido, personas protegidas, tono, plan)
 3. Verificar feeds RSS del municipio (`URL/feed` en el browser)
 4. Cargar fuentes en `fuentes/rss_feeds.txt`
 5. Pedirle a Antigravity/Cursor que genere el workflow desde el template
 6. Importar workflow en n8n
 7. Configurar credenciales (Google Sheets, Telegram, GitHub)
 8. Activar workflow y testear con 1 noticia
 9. Primera aprobación en Telegram con el cliente presente (demo en vivo)
-

PROMPT BASE DE GEMINI (parametrizado)

Este es el prompt que va en el nodo "Message a model" de n8n.

Las variables en [CORCHETES] se reemplazan por el config del cliente.

Eres el redactor de prensa de [MUNICIPIO], Argentina.
Tu cliente representa al [PARTIDO_POLITICO].

CONTEXTO POLÍTICO:

- Rol municipal: [OFICIALISMO|OPOSICION]
- Personas a proteger (NUNCA criticar): [PERSONAS_PROTEGIDAS]
- Temas prioritarios: [TEMAS_AMPLIFICAR]
- Temas vetados: [TEMAS_VETAR]
- Tono deseado: [TONO]

TAREA:

Del listado de noticias recibido, selecciona LA MÁS RELEVANTE considerando el perfil político anterior.

REGLAS DE SELECCIÓN:

1. Priorizar noticias que beneficien la imagen del cliente
2. Nunca seleccionar noticias que dañen a personas protegidas
3. Las noticias de emergencia (desapariciones, catástrofes, alertas de seguridad) SIEMPRE se incluyen independientemente del perfil político – solo cambia el encuadre
4. Las noticias nacionales en tendencia se incluyen o excluyen según la alineación política definida

OUTPUT: JSON estricto con este formato exacto:

```
{  
  "title_en": "título en inglés optimizado para SEO",  
  "slug": "slug-url-friendly-maximo-40-chars",  
  "content_markdown_en": "artículo completo en inglés con formato Markdown",  
  "approval_summary_es": "resumen en español para aprobar en Telegram (max 2 líneas)"  
}
```

CRITICAL OUTPUT RULE: Respond with STRICT JSON ONLY.

No markdown code fences. No text before or after the JSON.

No comments. Must start with { and end with }.

NOTAS TÉCNICAS IMPORTANTES

Sobre n8n y webhooks

- Los webhooks SOLO funcionan con el workflow en estado ACTIVO (toggle verde)
- En localhost, SIEMPRE necesita Cloudflare Tunnel o ngrok para recibir callbacks de Telegram

- Al reiniciar n8n, las variables de entorno del .env deben cargarse ANTES de iniciar n8n (usar iniciar_liberty.ps1)

Sobre la API de Instagram (el más complejo)

- Requiere cuenta de Creador o Empresa (no personal)
- Requiere Página de Facebook vinculada
- La imagen DEBE estar en URL pública ANTES de llamar a la API
- Flujo obligatorio de 2 pasos: crear container → publicar
- Límite: 25 publicaciones por día
- Token expira cada 60 días (necesita refresh automático)

Sobre Google Sheets como DB

- Todos los nodos de Sheets DEBEN usar la misma credencial (actualmente: "Google Sheets account 6", id: wIHYP05Knj4a6xCZ)
- El Sheet activo es: 1vvJkif8H1W1R2jQquUzmPCpeyvYJic6-_byQVng0eJ4
- Si hay error de autorización en Sheets, re-autenticar esa credencial en n8n → Credentials → Google Sheets account 6

Sobre la generación de imágenes (pendiente)

- NO usar IA generativa (DALL-E, Stable Diffusion) para clientes municipales porque no garantiza consistencia de marca (logos, colores institucionales)
- Recomendado: Bannerbear (templates + API, ~\$49/mes) o Puppeteer propio (HTML→screenshot, costo cero)
- La imagen debe estar en URL pública antes de llamar a Instagram API

Sobre el deploy en Railway

- Archivos preparados en: C:\Users\Ryzen 5\Desktop\n8n-railway\
- NO hacer deploy hasta tener primer cliente (el costo se cubre con el contrato)
- Al migrar a Railway, las credenciales de n8n NO se exportan con el JSON (hay que reconfigurarlas manualmente en la nueva instancia)

INFORMACIÓN DEL DESARROLLADOR

- **Nombre:** Andrés Carrizo
- **Ubicación:** Malargüe, Mendoza, Argentina
- **GitHub:** <https://github.com/andrescarrizo1>

- **Sitio demo:** <https://lalibertadavanzandohoy.pages.dev>
 - **Repo web:** <https://github.com/andrescarrizo1/lalibertad-web>
 - **IDE de automatización:** Antigravity (Gemini) para generación/modificación de JSON
 - **Herramienta de análisis:** Perplexity/Comet para research de fuentes
-

Documento generado: Mayo 2025 Versión del workflow activa:

Liberty_Advance_v3_CredsFix.json